16-720 Computer Vision

Assignment 3: Geometry in Motion

Fall 2026 Released October 27, 2026

This PDF is the authoritative assignment specification. The README explains setup; the notebook supplies executable examples. Read Sections 4–5 before beginning.

Item	Requirement
Release	October 27, 2026
Due	November 24, 2026, 11:59 pm ET (Gradescope)
Base score	100 points
Optional credit	Up to 10 points total, including showcase participation
Expected time	8–10 hours
Runtime	Google Colab; required work runs on CPU
Submit	Python code, completed notebook, LaTeX-generated PDF, Hero Result, and static preview

2. Overview

Images flatten the world. Geometry lets us reason backward: which image measurements could have come from the same scene, which motion explains two views, and when a transformation is trustworthy. The same ideas support panorama stitching, planar tracking, camera pose, stereo reconstruction, structure from motion, optical flow, and image stabilization.

In this assignment you derive the constraints behind homography estimation and differential motion, implement a robust planar-motion pipeline, and investigate where its geometric and photometric assumptions fail. Part A derives the mathematics, Part B implements estimation and alignment, Part C builds a tracked planar application from video you shoot yourself, and Part D measures where the pipeline holds up and where it breaks.

Your final artifact must show more than a successful overlay. It must make the estimated motion visible, expose at least one failure boundary, and communicate what the result does and does not establish about the three-dimensional scene.

The feature detector, descriptor, matcher, image pyramid, interpolation, warping, video input/output, and plotting code are supplied. You write the estimation and alignment mathematics that the rest of the pipeline depends on.

3. Learning objectives

After completing this assignment, you should be able to:

- derive the linear constraints used to estimate a planar projective transformation;
- normalize image coordinates and solve homogeneous least-squares problems using SVD;
- implement and diagnose RANSAC for robust homography estimation;
- derive and solve the Lucas–Kanade brightness-constancy system;
- distinguish geometric error, photometric error, and downstream visual quality;
- explain the assumptions behind homographies and differential image alignment;
- connect projective geometry to camera pose, epipolar geometry, stereo, and reconstruction; and
- communicate motion and geometric failure through a polished visual result.

4. Academic-integrity and generative-AI policy

The policy follows the work being done, not the file or chat window it happens in.

Work	Policy
Part A	Generative AI may not generate, explain, solve, check, or rewrite responses.
Part B required functions	Generative AI may not generate, complete, translate, explain, debug, or rewrite the five required functions in `student/pa3.py`.
Parts C, D, and E	Generative AI is permitted and encouraged, with disclosure and demonstrated comprehension.

Working with others. Talk to your classmates. Discussing the derivations, debugging strategies, and experimental design with other students is one of the best ways to learn this material, and we encourage it — as is coming to office hours early and often. The line is simple: share ideas, not artifacts. Everything you submit — derivations, code, figures, and text — must be produced by you, and you may not read or copy another student's solutions or let anyone copy yours. List everyone you worked with in your acknowledgments; collaboration is never penalized, but unacknowledged copying is an integrity violation.

You may always use course notes, textbooks, papers, conventional web resources, official library documentation, and course staff. Do not call an OpenCV homography estimator inside a required Part B function. Acknowledge every collaborator, outside resource, dataset, pretrained weight, and permitted AI use.

5. Shared conventions and allowed tools

- Point arrays have shape $(N, 2)$, order (x, y), and dtype `float64`.
- Images are (H, W) or (H, W, C) and are indexed `image[y, x]`.
- Pixel centers lie at integer coordinates.
- A reported homography maps source to destination: $\mathbf{x}_{\text{dst}} \sim H\mathbf{x}_{\text{src}}$.
- Warping samples inversely; interpolation and validity are separate operations.
- An invalid homogeneous scale becomes an invalid measurement, not a clipped point.
- No required function mutates its inputs or forms an explicit matrix inverse where a solve applies.
- The shared random seed is 16720. All required code runs on a free Colab CPU runtime.

All required functions live in `student/pa3.py`. Do not change their names, argument order, or return structure. The released constants `RANSAC_MAX_TRIALS` $= 5000$, `LK_MAX_ITERS` $= 50$, `LK_TOL` $= 10^{-3}$, `LK_MIN_VALID` $= 25$, and `LK_MAX_CONDITION` $= 10^6$ are part of the contract. Import them instead of retyping their values.

6. Part A — THINK: Constraints, Estimation, and Motion (25 points; 1.5–2 hours)

Complete Part A without generative AI. Typeset all work in LaTeX and show intermediate reasoning.

6.1 A1 — Derive the DLT constraints (9 points)

Let corresponding image points be $\mathbf{x} = [x, y, 1]^T$ and $\mathbf{x}' = [x', y', 1]^T$, related by a homography $\mathbf{x}' \sim H\mathbf{x}$, where

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}.$$

Unlike the row-major ordering used in lecture, this problem uses the **column-major** vectorization

$$\mathbf{h} = \text{vec}_c(H) = [h_{11}, h_{21}, h_{31}, h_{12}, h_{22}, h_{32}, h_{13}, h_{23}, h_{33}]^T.$$

Use this ordering throughout A1 and in the released implementation. Do not transpose H; only its vectorization convention has changed.

1. Starting from homogeneous equality up to scale, eliminate the scale and derive two independent scalar equations contributed by one correspondence.
2. Collect coefficients according to $\text{vec}_c(H)$ and write the resulting 2×9 matrix A_i satisfying $A_i\mathbf{h} = \mathbf{0}$.
3. Let $\mathbf{h}_r = \text{vec}_r(H)$ denote the row-major vector used in lecture. Define a permutation matrix P by $\mathbf{h}_r = P\mathbf{h}$. Without listing all 81 entries of P, state how its nonzero positions are determined and derive the relationship between the column-major design matrix A and the lecture's row-major design matrix A_r.
4. State the minimum number of point correspondences needed in the nondegenerate case, explain why a homography has eight rather than nine degrees of freedom, and explain why four collinear correspondences are degenerate.
5. For noisy data, formulate the total least-squares problem and explain why the right singular vector of A associated with its smallest singular value gives the DLT solution. Explain how this vector must be reshaped to recover H without accidentally transposing it.

6.2 A2 — Normalize a small correspondence set (5 points)

Consider image points $(0, 0)$, $(2, 0)$, $(2, 2)$, and $(0, 2)$.

1. Compute their centroid and mean Euclidean distance from the centroid.
2. Construct a similarity transform T that moves the centroid to the origin and makes the mean distance equal to $\sqrt{2}$.
3. Transform all four points explicitly.

4. If normalized DLT estimates $\widetilde{H}$ from $\widetilde{\mathbf{x}} = T\mathbf{x}$ and $\widetilde{\mathbf{x}}' = T'\mathbf{x}'$, derive the expression that converts $\widetilde{H}$ back to image coordinates.
5. Explain why coordinate normalization improves the numerical conditioning of the DLT system. Does it make the least-squares DLT estimator robust to mismatched correspondences? Justify your answer.

6.3 A3 — RANSAC sampling without replacement (5 points)

The lecture analysis uses the approximation that points in a minimal sample are independent draws with a fixed inlier probability. For this problem, instead assume that each RANSAC trial selects s correspondences uniformly without replacement within that trial. Suppose the data contain exactly M correspondences, of which exactly I are inliers.

1. Write the exact probability q that one size-s sample contains only inliers. Express it using binomial coefficients or as a product that makes sampling without replacement explicit.
2. Use the following stopping-budget formula; you do not need to derive it:

$$N = \left\lceil \frac{\log(1-p)}{\log(1-q)} \right\rceil .$$

   For homography estimation, compute N for $M = 40$, $I = 20$, $s = 4$, and $p = 0.99$. Compare it with the budget obtained using the lecture approximation $q_{\text{iid}} = (I/M)^s$, and briefly explain the direction of the difference.
3. After several trials, the best model has 26 inliers among the same 40 correspondences. Should an adaptive RANSAC implementation increase or decrease its stopping budget? Explain. Why could an overly optimistic consensus count make the algorithm stop too early?
4. Give one example in which RANSAC returns a high-consensus homography that is nevertheless the wrong model for the intended scene.

6.4 A4 — Local motion with additive brightness change (6 points)

Suppose a small patch undergoes a translation (u, v) and an unknown constant additive brightness change β between two frames:

$$I(x + u, y + v, t + 1) = I(x, y, t) + \beta.$$

1. Use a first-order Taylor approximation to derive $I_x u + I_y v - \beta = -I_t$. Explain how ordinary brightness constancy is recovered as a special case.
2. Explain why one pixel cannot determine all three unknowns. State what spatial variation a patch must contain for the joint system to be well constrained.
3. For a patch with gradient rows $(I_x, I_y) = (1, 0), (0, 1), (1, 1)$ and temporal derivatives $I_t = -1, -2, -4$, construct $A\boldsymbol{\theta} = \mathbf{b}$ for $\boldsymbol{\theta} = [u, v, \beta]^T$ and solve exactly.
4. Describe what the eigenvalues and conditioning of $A^T A$ reveal about ambiguity among horizontal motion, vertical motion, and an additive brightness change.
5. Predict which method should fail first as inter-frame displacement increases: single-scale Lucas–Kanade refinement or descriptor matching with RANSAC. Revisit this prediction in Part D.

7. Part B — BUILD: Robust Planar Motion (35 points; 2.5–3 hours)

You may use NumPy, the supplied helpers, OpenCV documentation, SciPy documentation, and course materials, but generative AI may not write the required implementations. The starter code provides image I/O, feature detection and description, candidate matching, visualization, deterministic test data, video decoding, and Colab setup.

7.1 B1 — Read the geometry pipeline (5 points)

Before implementing anything, inspect the numbered cells of the starter notebook and answer the following. Each answer must name a specific array shape, coordinate convention, or state transition, not only a general definition.

1. **B1.1** In the coordinate-convention cell, what shapes do `xy` and `to_homogeneous(xy)` have, which columns hold x and y, and why does projection transpose the row array around the matrix multiplication?
2. **B1.2** The supplied warp takes `H_src_to_dst`. Which direction does that matrix report, which direction is solved for each destination pixel, and why is that not a contradiction?
3. **B1.3** What are the two outputs of `bilinear_sample`, and why must Lucas–Kanade combine validity masks rather than treat interpolation as a validity test?
4. **B1.4** Identify the two rows of the DLT scaffold that match your A1 equations. Why must the recovered vector be reshaped with `order="F"`?

5. **B1.5** The pyramid list holds full resolution at index zero. Trace a full-resolution displacement of eight pixels down to level two and back.

7.2 B2 — Normalized DLT (10 points)

Implement the three estimation functions:

- `normalize_points(xy)` returns the normalized homogeneous points, whose last coordinate is exactly one, and the 3×3 similarity transform that moves the centroid to the origin and sets the mean Euclidean distance to $\sqrt{2}$. Coincident points are an error, not a division by zero.
- `estimate_homography_dlt(src_xy, dst_xy)` builds A using the column-major convention from A1, solves by SVD, reshapes without transposing the intended transform, denormalizes, and returns a consistently scaled 3×3 homography. Reject a design matrix whose numerical rank is below eight.
- `symmetric_transfer_error(H, src_xy, dst_xy)` returns, for each correspondence i,

$$e_i = \tfrac{1}{2}\left(\|\pi(H\mathbf{x}_i) - \mathbf{x}'_i\|_2^2 + \|\pi(H^{-1}\mathbf{x}'_i) - \mathbf{x}_i\|_2^2\right),$$

  in units of pixel squared, where π maps homogeneous to Cartesian coordinates.

An invalid projection scale or a noninvertible map gives `inf` for the affected entries, never `nan` and never a clipped coordinate. Multiplying H by any nonzero scalar must leave every e_i unchanged. Scale the returned H so that `H[2,2]` equals one when that entry is safely nonzero; otherwise return the unit-Frobenius-norm representative whose largest-magnitude entry is positive.

Your implementation must pass tests for exact synthetic transforms, noisy overdetermined data, input order, scale invariance, input mutation, accidental transpose, and degenerate configurations. You may use `numpy.linalg.svd`; you may not call an OpenCV homography-estimation function inside these functions.

7.3 B3 — RANSAC homography estimation (9 points)

Implement `ransac_homography(src_xy, dst_xy, threshold, confidence, rng)`, which returns `(H, inlier_mask, info)`. The threshold is in pixels and is squared before comparison with the B2 error. Each trial must:

- sample four distinct correspondences without replacement;
- reject repeated points and any collinear triplet in either minimal set;
- score all correspondences using symmetric transfer error;
- update the exact finite-population budget using $\binom{k}{4}/\binom{N}{4}$ for the current consensus size k;
- stop at that budget or `RANSAC_MAX_TRIALS` trials, whichever comes first;
- break ties by larger consensus, then lower median inlier error, then lower total inlier error, then earlier trial; and
- re-estimate the final homography from every returned inlier.

Use only the generator passed in `rng`, never the global random state. The same inputs and the same seed must give the same output.

`inlier_mask` is a boolean array of shape $(N,)$. `info` contains at least `trials` (an `int`, counting rejected degenerate draws), `adaptive_budget` (an `int`), `best_sample` (a tuple of four `int`, so that two runs compare with ==), and `errors` (a `float64` array of shape $(N,)$ holding each correspondence's error under the returned H). If the budget is exhausted with no four-point model reaching a four-correspondence consensus, raise `RuntimeError`; the supplied Part C cell catches that case and marks the frame invalid.

Evaluate direct DLT and RANSAC on the supplied 100-point, 640×480 synthetic scenes with noise $\sigma = 1$ px, outlier fractions 0, 0.2, 0.4, and 0.6, and seeds 16720 through 16724, using a 3-pixel threshold and confidence 0.99. Report median corner-transfer RMSE, inlier precision and recall, and runtime. The performance score is diagnostic; most credit is for correctness and interpretation.

7.4 B4 — Coarse-to-fine Lucas–Kanade refinement (11 points)

Implement `lucas_kanade_refine(template, image, p0, levels)`, which returns `(p, history)`.

Single-scale Lucas–Kanade linearizes image alignment near a current estimate, so it generally requires the remaining displacement to be small. A coarse-to-fine implementation reduces a large full-resolution displacement before applying that same local update. The notebook provides warping, interpolation, gradients, and Gaussian-pyramid construction. Use the following procedure, where level 0 is full resolution and level $L-1$ is the coarsest level.

1. Construct corresponding pyramids T^ℓ and I^ℓ using the supplied helper. Each successive level is smaller by a factor of two in each spatial dimension.

2. Write the translation at level ℓ as $p^\ell = [u^\ell, v^\ell]^T$. Convert the initial full-resolution translation $p_0 = [u_0, v_0]^T$ to the coarsest coordinates: $p^{L-1} = p_0 / 2^{L-1}$.
3. At the current level ℓ, warp I^ℓ toward T^ℓ using the current translation. Retain only pixels whose warped coordinates are valid. For each valid pixel x, form

$$r(x) = T^\ell(x) - I^\ell(x + p^\ell), \qquad A(x,:) = \begin{bmatrix} I_x & I_y \end{bmatrix}_{x+p^\ell}.$$

4. Solve the overdetermined system $A\,\Delta p \simeq r$ by least squares, update $p^\ell \leftarrow p^\ell + \Delta p$, and repeat until the update norm falls below `LK_TOL` level pixels or `LK_MAX_ITERS` iterations have run. Do not form $(A^T A)^{-1}$ explicitly.
5. End a level early when fewer than `LK_MIN_VALID` pixels are jointly valid, when the least-squares rank is below two, or when the design matrix's condition number exceeds `LK_MAX_CONDITION`. Record which of these occurred.
6. If $\ell > 0$, initialize the next finer level with $p^{\ell-1} = 2p^\ell$ and repeat the refinement. The final answer is p^0 in full-resolution pixels.

`history` is a list of records tracing the search. It starts with the coarsest `level_start` record and ends with the level-zero `level_end` record. Every record holds `phase` (one of `level_start`, `iteration`, `level_end`), `level` (zero is full resolution), `p_level` (the estimate in that level's pixels), `p_full` (the same estimate times 2^ℓ), and `status` (one of `start`, `iterate`, `converged`, `max_iterations`, `singular`, `ill_conditioned`, `insufficient_overlap`). Iteration records also hold `photometric_rmse`, the root-mean-square residual over that iteration's valid pixels. You may add more keys. These are the graded minimum, and the notebook cells read them by name.

As intermediate checks, confirm that a displacement of eight pixels at full resolution becomes two pixels at pyramid level 2; verify the residual sign using a one-pixel synthetic translation; and report the estimate before and after refinement at every level. A single-scale run is the same algorithm with $L = 1$.

Verify the method on synthetic translations and on the checksum-verified Oxford VGG `wall` sequence, fetched with `python data/setup_data.py --b4`. For the graded sequence row, use `wall/img1.ppm` to `wall/img2.ppm` with `wall/H1to2p`, start from $p_0 = (0, 0)$, and take the half-open 64×64 region `x=68:132, y=268:332` from both frames. The reference endpoint is the displacement of the region's center under the released homography. Report the endpoint error and the corner RMSE between the four translated region corners and their homography-warped positions. Plot the fixed region and patch, one- versus four-level photometric error by iteration, the reference and predicted quadrilaterals, and both geometric errors. A clearly labeled synthetic fallback keeps the notebook runnable offline but earns no credit for this row.

Compare single-scale and coarse-to-fine refinement, and clearly separate failure caused by poor initialization from failure of brightness constancy. B4 is a separate translation study: it assumes $\beta = 0$, so the A4 brightness parameter is not a hidden implementation requirement.

8. Part C — CREATE: Geometry-in-Motion Experience (20 points; 2.5–3 hours)

You may use generative AI for coding, debugging, refactoring, visualization, interface work, and integration with approved pretrained matchers or flow models. You remain responsible for understanding, testing, and modifying everything you submit.

8.1 C1 — Choose a planar visual target and application (4 points)

Using your own phone or camera, capture a 5–15 second video containing a clearly visible planar target such as a poster, book cover, sign, painting, tabletop card, or building facade. This self-captured sequence is required. Downloaded and supplied sequences, including the clips under `data/fallback/`, may be used freely while you develop and debug, but they do not satisfy Parts C or D. The one exception is described in Section 10.

Choose one application:

- a tracked augmented-reality overlay that remains attached to the plane;
- stabilization that holds the chosen plane fixed while the camera moves; or
- another application approved by the course staff that visibly depends on planar motion estimation.

Document the target, camera used, capture location and setup, camera motion, lighting, frame count and resolution, and why a homography is a plausible model. Include one still frame from the raw capture and state one anticipated violation of the model.

8.2 C2 — Build and improve the system (10 points)

Build the per-frame baseline from the supplied components: up to 2000 SIFT features, L2 two-nearest-neighbor matching with ratio 0.75, a mutual-consistency check, your normalized-DLT RANSAC at a 3-pixel threshold and confidence 0.99, and

a processing resolution of at most 720 pixels on the long edge. That cap limits the longer side in pixels. It is not the 720p limit that applies to the submitted artifact.

Your baseline estimates a target-to-frame homography for each frame. The notebook gives you the per-frame estimator, a default overlay renderer, and a smoke demonstration you can run. You build the temporal state around them.

A frame is valid only when its map is finite and invertible, has at least 10 inliers, an inlier ratio of at least 0.25, target-source inlier coverage of at least 0.15, a median symmetric error of at most 3 pixels, and a finite convex projected quadrilateral. A stale result may be held for at most three frames with a visible `STALE` label; after that, hide it and attempt ungated global reinitialization. Display a scalar confidence built from inlier ratio, error, and spatial coverage.

Make at least one substantive improvement beyond the baseline, such as:

- combining feature-based homography estimation with photometric refinement;
- detecting tracking failure and re-initializing automatically;
- smoothing motion without producing unacceptable lag;
- handling partial occlusion using spatially distributed inliers;
- replacing the supplied feature matcher with an approved learned matcher; or
- comparing a classical tracker with a modern correspondence or flow system.

The final pipeline must produce both a clean application view and a diagnostic view showing correspondences and inliers, the tracked quadrilateral, confidence or error over time, and the current validity state.

Long clips run RANSAC once per frame, and a frame with no valid model costs the full trial budget. If your capture is slow to process, decimate it with `read_video(..., frame_stride=k)` or cache the target's features once with `extract_sift` and call `match_sift` per frame. Neither changes the required geometry.

8.3 C3 — AI-comprehension reflection (6 points)

In approximately 200–300 words, answer:

1. What did generative AI help you implement or communicate?
2. How does the most important AI-assisted portion work, including its inputs, outputs, and assumptions?
3. What did you change, debug, test, or redesign from the initially generated version, and why?
4. Identify one check that would detect a plausible-looking but geometrically incorrect result.

If you used no generative AI, say so plainly and use questions 2 and 3 to analyze the most important component you wrote yourself or obtained from a permitted third party, and the design iterations you went through. Work done without AI receives equal credit.

9. Part D — UNDERSTAND & SHOW: Evidence, Failure, and Communication (20 points; 1.5–2 hours)

Generative AI is permitted and encouraged for visualization, with disclosure.

9.1 D1 — Controlled robustness study (7 points)

Use your Part C clip as the baseline, then record at least three short variants in the same session, changing one factor at a time: viewpoint, motion blur, illumination, partial occlusion, repeated texture, parallax or nonplanarity, reduced texture, or larger inter-frame displacement. For each condition:

- describe exactly what you held fixed and what you changed, in enough detail for another student to repeat the capture;
- state a prediction before running the experiment;
- manually annotate the target's four ordered corners on 5–10 representative frames;
- report mean corner error or quadrilateral IoU, plus a tracking metric such as valid-frame rate;
- show one representative visualization; and
- explain the result using sampling, conditioning, consensus, brightness constancy, the aperture problem, or model mismatch.

Include a compact capture table listing the target, camera motion, distance or viewpoint, lighting, controlled variable, duration, frame count, and resolution for the baseline and each condition. The purpose is to generate evidence from a physical setup you designed, rather than only assembling code around an existing video.

At least one study must compare direct DLT with RANSAC, and at least one must compare single-scale with coarse-to-fine Lucas–Kanade. Revisit the prediction from A4. Inlier residual alone measures self-consistency, not external geometric truth; that is why the manual corner annotations are required.

9.2 D2 — When a good-looking warp is wrong (5 points)

Select one success and one failure from your video. For each, show the input frame, inliers, projected target boundary, application output, and synchronized error or confidence trace. Explain:

1. whether the dominant problem is measurement noise, outliers, optimization, interpolation, or an invalid scene model;
2. why visual plausibility alone is insufficient evidence of geometric correctness; and
3. what additional camera, epipolar, stereo, or multi-view information would be required to support a genuinely three-dimensional interpretation.

9.3 D3 — Hero Result (8 points)

Create one polished visual artifact: a 10–30 second video, animated comparison, or compact interactive webpage. It must remain understandable without reading your code and include:

- the original view and final application result;
- a synchronized diagnostic view of estimated geometry;
- a clear success and an informative failure or recovery;
- one before/after intervention that improves behavior;
- a concise statement of the assumptions being made; and
- one honest statement of what the result does *not* establish.

Accepted formats are H.264 MP4 (yuv420p), GIF, or a self-contained HTML file, at most 30 seconds, 720p, 30 fps, and 75 MB. Also submit one static preview frame, at least 1280 pixels on its longer side where the source allows. Choose the single frame, panel, or short moment that best communicates the project as your **Hero Result**; it should be legible outside the notebook and suitable for a course showcase or portfolio.

10. Capture access, privacy, safety, and accessibility

Do not record non-consenting people, private documents, personal information on screens, unsafe behavior, or prohibited locations. Do not take risks to obtain a more dramatic result. Keep raw captures on your own machine and submit only the evidence requested.

The course-authored package under `data/fallback/` may replace the act of personal recording, and only after written, advance approval from course staff for an access, privacy, safety, or accessibility barrier. Request approval before starting Part C, and no later than seven calendar days before the deadline once you know the barrier exists. Follow `data/fallback/README.md`: contact the instructor and the PA3 lead privately, name only the applicable category and the requirement it affects, and wait for an approval identifier of the form `PA3-FB-YYYY-NNN`. You do not need to state a diagnosis or any other personal detail. Enter the identifier in the notebook's approval guard, and record only the identifier, approval date, and approving staff initials in your acknowledgments.

Approval waives personal recording and nothing else. Every implementation, prediction, controlled condition, manual annotation, metric, comparison, analysis, Hero Result, reflection, and acknowledgment still applies. Without a valid identifier the supplied clips stay debugging material.

11. Part E — EXPLORE: Optional extension (up to +10 points; about 1 hour)

Generative AI is permitted with disclosure. Do something we did not explicitly ask you to do. Extend the method, improve it, break it, compare it, or apply it somewhere unexpected. Possibilities include:

- recover an essential or fundamental matrix and visualize epipolar lines;
- perform calibrated stereo and produce a disparity or depth visualization;
- reconstruct and display a sparse 3D point cloud from two or more views;
- compare Lucas–Kanade or Horn–Schunck with a modern learned flow model;
- implement inverse-compositional alignment and measure its speed advantage;
- add a nonplanarity or parallax detector, or a piecewise-planar model; or
- create an especially compelling AR, stabilization, or geometry-teaching interface.

A meaningful additional exploration with a clear result and short explanation earns up to $+5$. A particularly thoughtful, technically interesting, creative, or ambitious exploration earns up to $+10$ in total for Part E.

Every student must create a Hero Result, but public participation is optional. Students who choose to participate may post their final artifact to the announced course showcase channel by the stated deadline. Participation earns $+1$ extra-credit point. That point and any Part E award share the same 10-point cap, so the combined award is $\min(10,\ \text{Part E points} + 1)$. Voting never affects academic credit, and public posting is never required.

12. Submission specification

Submit the following through the course submission system:

- completed Colab notebook and required source files;
- a LaTeX-generated PDF containing all written responses;
- required plots, tables, correspondence views, and diagnostic visualizations;
- the Geometry-in-Motion artifact and a static preview of its Hero Result;
- the AI-comprehension reflection; and
- acknowledgment of collaborators, outside resources, data, pretrained weights, and AI assistance.

All required code must execute in Google Colab on readily available course hardware. Do not submit private API keys, authentication tokens, identifiable private imagery, or needlessly large raw videos.

Use the provided answer-template `.tex` file; do not retype the questions or change their labels. Compile and upload one PDF containing all written answers. Begin each labeled subproblem on a new page using the included page breaks. Assign every answer page to the corresponding Gradescope question and verify the rendered preview.

13. Point summary

Part	Points
A1	9
A2	5
A3	5
A4	6
B1	5
B2	10
B3	9
B4	11
C1	4
C2	10
C3	6
D1	7
D2	5
D3	8
Base total	**100**
E and showcase combined	**0 to +10**